Phase 1: Assisted Development

Module 2: Context Development

Building and managing a context framework

👋 Agenda

- Learning objectives
- Prompt vs. Context Engineering
- Context Framework Contents
- Configuring the entry point
- Three Context Tips
- Four Collaboration Tips
- Context Framework Examples
- AI Tool Landscape
- Legal, Ethical & IP Considerations
- The evolution of our Config Service

📚 Learning Objectives

- **Goal**: Build a usable context framework for a web API and its UI
- **Takeaway**: The beginnings of a context framework for future use
- **Hope**: You will use this context framework in the future

Along the way you will:

1. **Collaboratively** craft context documents with your assistant
2. **Take control** of when context is brought into short-term memory
3. **Compare its value** and cost with not using structured context

Prompt vs. Context Engineering

- Recognises the real value of our solutions is in the structured communications that surrounds them
- Context, unlike a prompt, captures our conversations, plans, and refinements that influence the implementation
- Prompts tend to be ephemeral and do not become part of the implementation

> Sean Grove, OpenAI: Prompt Engineering is Dead (talk)
> "A *written specification* aligns humans"

📦 Context Framework Contents

> Can be global (cross-project) or defined locally to a project

Current focus:

- **About**: Project description, problem it's solving, user personas, constraints
- **Architecture**: Patterns, module relationships, data flow, persistence
- **Implementation**: Languages, versions, external dependencies, preferences

Next, in Module 3:

- **Planning**: high-level plan, current status, task breakdown
- **Tooling**: environment details, CLI tools

⚙️ Configuring the entry point

- All AI coding tools have a mechanism to auto-load context
  - Often called "rules", but increasingly moving toward `AGENTS.md`
- Use this mechanism to load additional files
  - Potentially as needed
- We will use `.clinerules/memory.md`
- `AGENTS.md` has emerged as a standard

Context Frameworks Examples

- memory-bank from Cline
- stdlib from Geoff Huntley
- AgentOS from Brian Casel
- Spec Kit from GitHub (*can recommend*)
- BMad Method from Brian Madison

💡 Context Tip #1

Clean context is happy context

Clean the window when it contains:

- **Incorrect** information
- **Missing** important details
- **Noisey** or irrelevant details

💡 Context Tip #2

Interesting combination of syntax

```
<conditional-block context-check="general-formatting">
IF this General Formatting section already read in current context:
  SKIP: Re-reading this section
  NOTE: "Using General Formatting rules already in context"
ELSE:
  READ: The following formatting rules

## General Formatting


...
```

Your assistant will know what you mean.

💡 Context Tip #3

Your CFx will be as familiar as what you're building

- If you're building something for the first time, expect to start your context framework from [mostly] scratch - and that's okay
- If you already have existing assets (docs, code, etc), use that to collaboratively build your context framework
- Expect to iterate on it more than you expect 😉

💡 Collaboration Tip #1

Role + Action + Context = Good Results

Be clear about

- **Who** you want the AI to be (perspective)
- **What** you want the AI to do (scope)
- **With** the assets you provide (context)
- Role: coding assistant

Example:

> "As as a senior Python dev, refactor the EventHandler class in @events/EventHandler.ts to handle its events asynchronously using the queue."

💡 Collaboration Tip #2

Don't assume you're right, validate your assumptions

- Use your assistant as a skeptical critical thinker
- You don't want fragile/unvalidated ideas, challenge them
- Leave yourself 15% of uncertainty as space to be surprised
- Role: skeptic/neckbeard 😂

Example:

> "What are 3 considerations of the caching strategy used by @user_preferences.py that could make it a poor choice?"

💡 Collaboration Tip #3

Plan before you act

- Greatly increases precision (across *all* models)
- Reduces the difference between what you expected and what you got
- Great at creating examples when you don't already have them
- Role: planner

Example:

> "Without making any filesystem changes, please describe the steps you would take to fix all of the type checking errors and warnings."

💡 Collaboration Tip #4

Ask your assistant to explain themselves

- You *MUST* read and understand all of YOUR code
- So if you see something you don't understand ... just ask
- These prompts are particularly useful when using unfamiliar languages & frameworks
- Role: mentor

Example:

> "Please explain why you used the `Protocol` type in @socket_handler_interface.py instead of an ABC? And what does `...` mean?"

🏝️ AI Tool Landscape

Categories

- **IDE integrations** (Cursor, Cline, RooCode, Windsurf)
- **Standalone assistants** (Claude Code, ChatGPT)
- **Specialized tools** (pipeline integrations, PR tools)

Selection Criteria

- Cost
- Integration requirements
- Precision, and accessibility
- Model options per feature (BYO?)

🚀 Let's Begin

1. Start with `memory/ABOUT.md`
2. Auto load it using `.clinerules/memory.md`
3. Test your assistant's understanding